

L5 Feature engineering + Model evaluation and choice

Dr. Zixin Zhong

Data Science and Analytics Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

February 28, 2026

Outline

1 Feature engineering

- Polynomial transformation
- Boolean, nominal, ordinal
- Missing values
- Nonlinear transformations

2 Model evaluation and choice

- Generalization of supervised learning algorithms
- Error metric, regularizer, no free lunch!
- Generalization, overfitting
- Data partition: train, test and validation



Recap of Lecture 4

- Logistic regression
- SVM
 - Linearly separable, γ -linearly separable
 - Margin, minimum margin, geometric margin
 - SVM: maximum margin linear classifier, unique solution
 - SVM: offset, slack, leave-one-out cross-validation error



Summary of logistic regression

- Given the logistic regression probabilistic model

$$\Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = g(y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0)) = \frac{1}{1 + \exp(-y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0))},$$



Summary of logistic regression

- Given the logistic regression probabilistic model

$$\Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = g(y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0)) = \frac{1}{1 + \exp(-y_t (\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0))}, \quad (0.1)$$

- Take the gradients of

$$\log [1 + \exp(-y_t (\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))]$$

with respect to $\boldsymbol{\theta}$ and θ_0 separately and update parameters $\boldsymbol{\theta}$ and θ_0 ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t [1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)] \quad (0.2)$$

$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t [1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)]. \quad (0.3)$$

- Stochastic gradient descent: update $\boldsymbol{\theta}$ and θ_0 until the termination condition is met.



Summary of logistic regression: regularization

- ⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^n \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^n \frac{1}{1 + \exp(-y_t(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0))}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
 - ▶ If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t . By scaling up the parameters, we make these values larger and larger.
- Regularization: e.g., ℓ_2 -norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^n \log [1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))]. \quad (0.4)$$



Understand θ^* and θ_0^* (optional)

⊙ Vector of **optimal** prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \theta^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\theta}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

⇒ There is **no further linearly available information** in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to **improve** the prediction probabilities (or mistake probabilities).

Theorem 0.1

At the optimal solution for the parameter and offset vector θ^, θ_0^* , the prediction error vector $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \theta^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the data, i.e., for any $\bar{\theta}, \bar{\theta}_0$,*

$$\langle \mathbf{z}, \mathbf{e} \rangle = 0, \quad \text{where} \quad \mathbf{z} := (\langle \bar{\theta}, \mathbf{x}_t \rangle + \bar{\theta}_0)_{t=1}^n. \quad (0.5)$$

*Furthermore if a prediction vector $\mathbf{e}$ satisfies (0.5), it has the **minimal ℓ_2 norm** of $(\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \theta, \theta_0))_{t=1}^n$ over all θ, θ_0 .*

Hint for proof: Assume $\langle \mathbf{z}, \mathbf{e} \rangle \neq 0$, we can prove $\mathbf{e}' := \mathbf{e} - \frac{\langle \mathbf{z}, \mathbf{e} \rangle}{\langle \mathbf{z}, \mathbf{z} \rangle} \mathbf{z}$ satisfies $\|\mathbf{e}'\| < \|\mathbf{e}\|$.



Summary of logistic regression: multi-class classification

- ⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp(-(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0))} \quad (0.6)$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^M) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^M \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \quad (0.7)$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

- Without loss of generality, we can assume that **one of the classes (say the first one)** has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$, which is useful for showing that (0.6) is a special case of (0.7).
 - **Why?**



Summary of logistic regression: further exploration

- See the impact of regularization parameter λ
- Try it for multi-class classification

- ◉ Find some dataset from Kaggle:

<https://www.kaggle.com/code/rtatman/datasets-for-regression-analysis>

- ◉ Useful Python packages and functions

`sklearn.linear_model.LogisticRegression`

`numpy.random`

`matplotlib.pyplot.scatter`



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_\theta = \text{sign}(\theta^\top \mathbf{x})$ means

$$\begin{cases} \theta^\top \mathbf{x} > 0, y = +1 \\ \theta^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow y = \text{sign}(\theta^\top \mathbf{x}) \Leftrightarrow y\theta^\top \mathbf{x} > 0.$$

Definition 0.2 (Margin of classifier)

The **margin** of classifier $f_\theta = \text{sign}(\theta^\top \mathbf{x})$ on sample $(\mathbf{x}, y)$ is

$$y\theta^\top \mathbf{x} \text{ or } y\langle \theta, \mathbf{x} \rangle$$

- **Positive** margin: $(\mathbf{x}, y)$ is **correctly** classified by θ .
- **Negative** margin: $(\mathbf{x}, y)$ is **not correctly** classified by θ .
- **Bigger** margin: $(\mathbf{x}, y)$ is **more correctly** classified by θ .



Linearly separable

Definition 0.3 (Linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \dots, n\}$ is **linearly separable** if there exists some $\boldsymbol{\theta}$ such that

$$y_t = \text{sign}(\boldsymbol{\theta}^\top \mathbf{x}_t) \text{ or } y_t \boldsymbol{\theta}^\top \mathbf{x}_t > 0 \quad \forall t = 1, 2, \dots, n. \quad (0.8)$$

There is some hyperplane that (**strictly**) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min_{1 \leq t \leq n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$



γ -linearly separable

Definition 0.4 (γ -linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \dots, n\}$ is **γ -linearly separable** for some $\gamma > 0$ if there exists some $\boldsymbol{\theta}$ such that

$$y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle \geq \gamma \quad \forall t = 1, 2, \dots, n. \quad (0.9)$$

There is some hyperplane that (**strictly**) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min_{1 \leq t \leq n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$

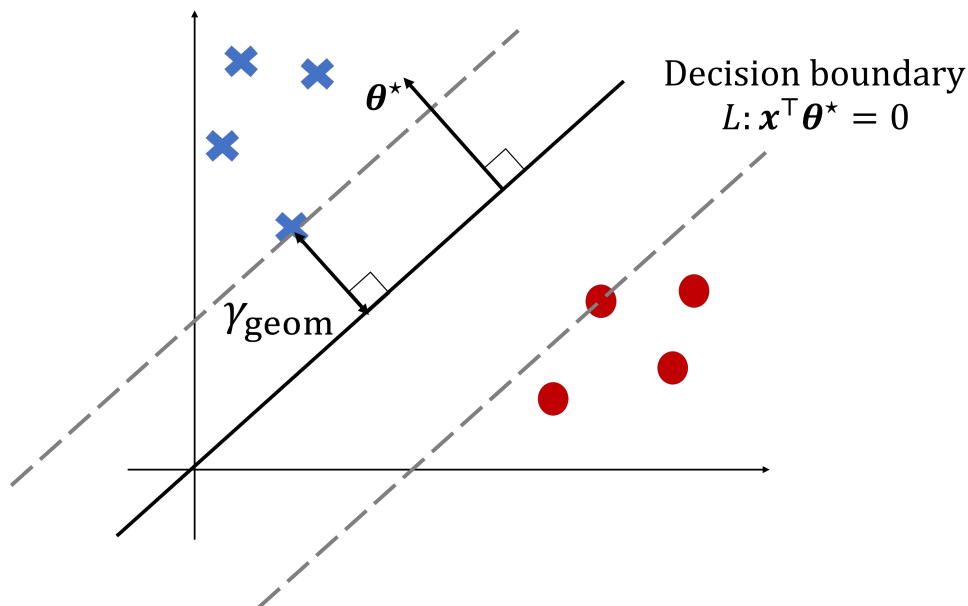


Geometric margin

- Geometric margin:

$$\gamma_{\text{geom}} := \frac{\gamma}{\|\boldsymbol{\theta}\|} = \frac{\min_{1 \leq t \leq n} y_t \mathbf{x}_t^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}.$$

- Smallest distance over all samples to the decision boundary $L = \{x \in \mathbb{R}^d : \langle \mathbf{x}, \boldsymbol{\theta} \rangle = 0\}$.
- $\gamma_{\text{geom}}^{-1}$: fair measure of the difficulty of the classification problem



Support Vector Machine (SVM)

- Maximize the geometric margin:

$$\max_{\boldsymbol{\theta}} \gamma_{\text{geom}} \text{ i.e. } \max_{\boldsymbol{\theta}} \frac{\min_{1 \leq t \leq n} y_t \mathbf{x}_t^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|} \quad (0.10)$$



- PRIMAL-SVM:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \|\boldsymbol{\theta}'\|^2 \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \dots, n \quad (0.11)$$



Support Vector Machine (SVM)

- The solution to PRIMAL-SVM is **unique**:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \|\boldsymbol{\theta}'\|^2 \quad \text{s. t.} \quad y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \dots, n$$

- **SVM algorithm**: find the maximum (geometric) margin linear classifier
- Why do we seek the maximum (geometric) margin linear classifier?
- Prove the solution to PRIMAL-SVM is **unique**?



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors θ_1, θ_2 ($\theta_1 \neq \theta_2$)
- Then $\|\theta_1\| = \|\theta_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\theta} = (\theta_1 + \theta_2)/2$
- Check feasibility:

Since $\forall t = 1, 2, \dots, n$, $y_t \mathbf{x}_t^\top \theta_1 \geq 1$ and $y_t \mathbf{x}_t^\top \theta_2 \geq 1$, by linearity of inner product, we have

$$y_t \mathbf{x}_t^\top \bar{\theta} = \frac{1}{2} y_t \mathbf{x}_t^\top \theta_1 + \frac{1}{2} y_t \mathbf{x}_t^\top \theta_2 \geq 1$$

$\Rightarrow \bar{\theta}$ is feasible



Primal-SVM: unique solution

Proof (continued):

$$\begin{aligned}\|\bar{\theta}\| &= \left\| \frac{1}{2} (\theta_1 + \theta_2) \right\| && \text{Triangle inequality} && \leq && \frac{1}{2} \|\theta_1\| + \frac{1}{2} \|\theta_2\| \\ & && && = && \|\theta_1\| = \|\theta_2\|\end{aligned}$$

Two case:

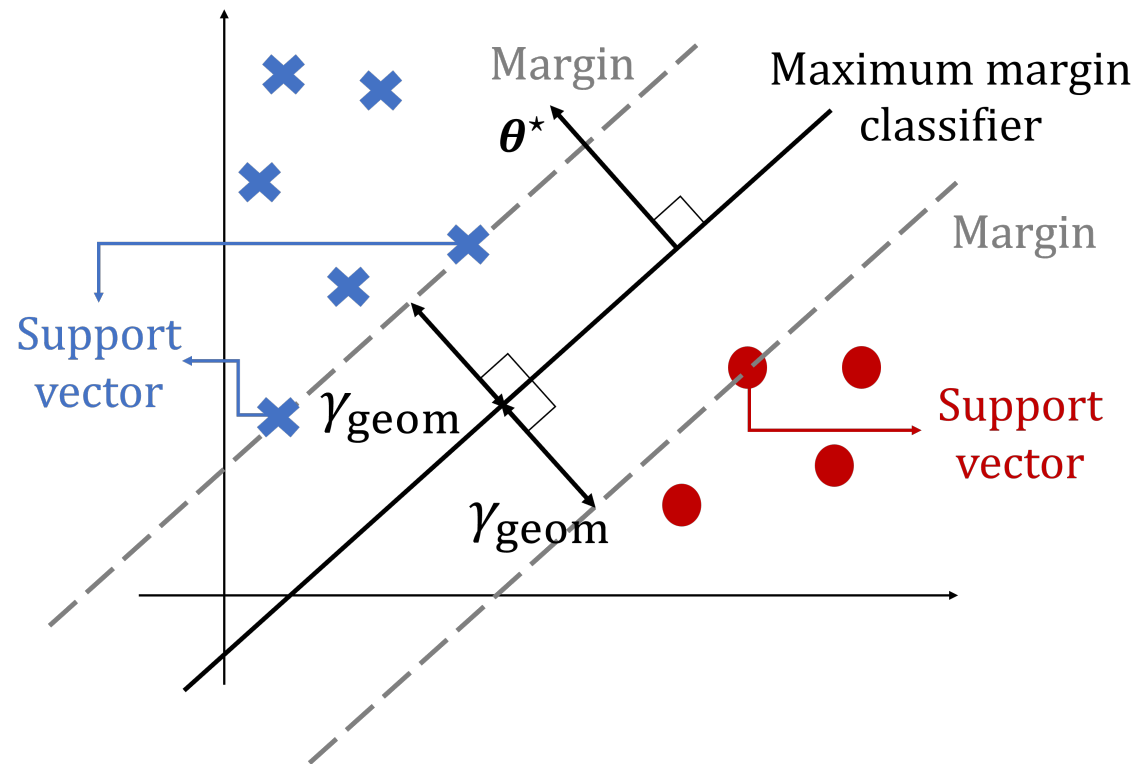
1. $\|\bar{\theta}\| < \|\theta_1\| = \|\theta_2\| \Rightarrow$ **Contradiction!**
2. $\|\bar{\theta}\| = \|\theta_1\| = \|\theta_2\| \Rightarrow$ Triangle inequality holds with equality. This can only happen if θ_1 is in the same direction as θ_2 .

In conclusion: there DO NOT exist 2 distinct min length solution vectors to PRIMAL-SVM.



Support Vector + Machine

- **Support vector** $\Leftarrow$ come from the input data set
influence the position of the max-margin classifier
- **Machine**: Input $\Rightarrow$ Machine $\Rightarrow$ Output classifier

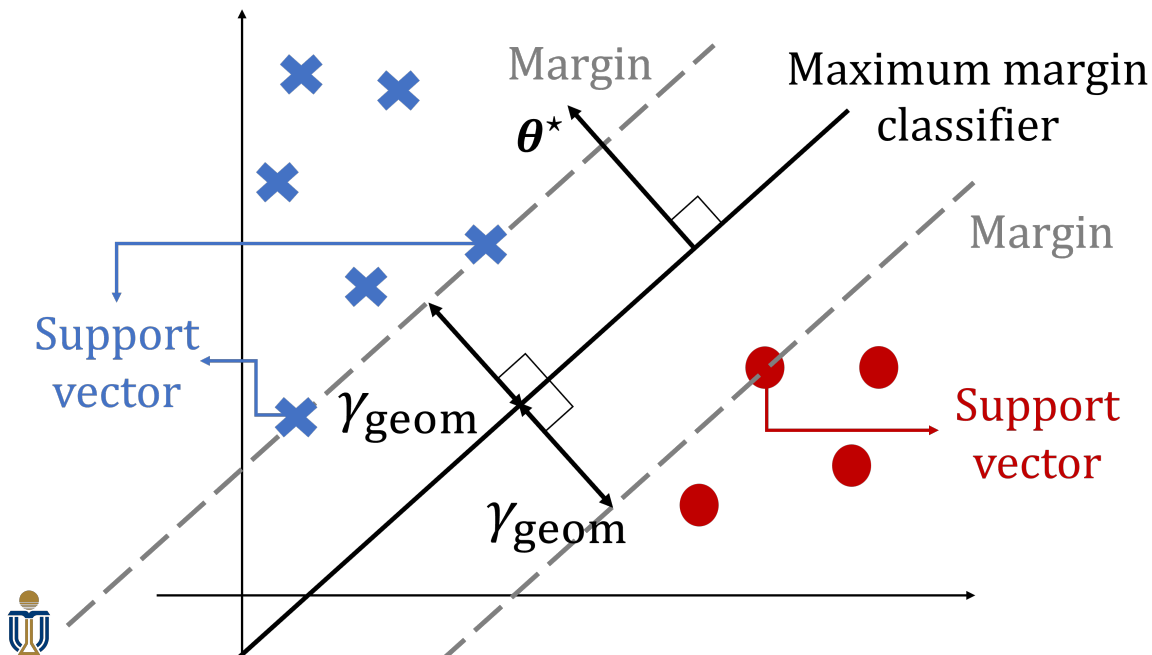


Robustness of max margin classifier

- As we have mentioned: larger margin $\Rightarrow$ more robust
- The (PRIMAL-SVM WITH OFFSET) problem:

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{1}{2} \|\boldsymbol{\theta}\|^2 \quad \text{s. t.} \quad y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall t = 1, 2, \dots, n$$

- Solution depends on a small number of data samples $(\mathbf{x}_t, y_t)$



Support vectors:

- Samples exactly on the margin.
- Quantify the quality of this binary classifier.



Offset and slack

(PRIMAL-SVM WITH OFFSET AND SLACK)

$$\begin{aligned} \min_{\boldsymbol{\theta}, \theta_0, \boldsymbol{\xi} \in \mathbb{R}^d \times \mathbb{R} \times \mathbb{R}_+^n} \quad & \frac{1}{2} \|\boldsymbol{\theta}'\|^2 + C \sum_{t=1}^n \xi_t \\ \text{s. t.} \quad & y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1 - \xi_t, \quad \xi_t \geq 0 \quad \forall t = 1, 2, \dots, n \end{aligned}$$

$\Longleftrightarrow$

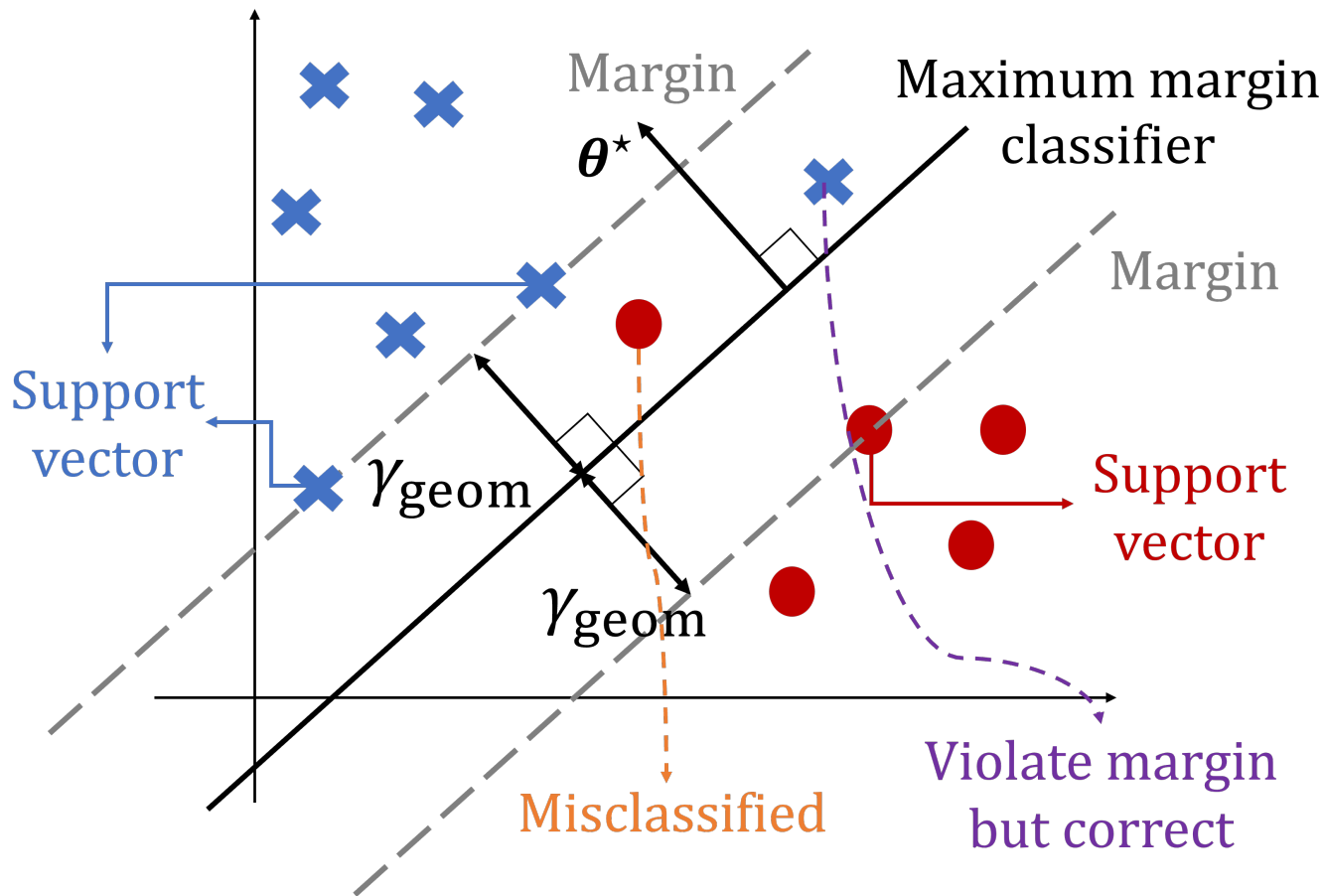
$$\min_{\boldsymbol{\theta}, \theta_0 \in \mathbb{R}^d \times \mathbb{R}} \frac{1}{2} \|\boldsymbol{\theta}'\|^2 + C \sum_{t=1}^n \underbrace{\left[1 - y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0)\right]^+}_{:=\xi_t}$$

where $f(z) = (1 - z)^+ = \max\{0, 1 - z\}$.

- Why offset? Why slack?
- How did we come to this point?



Allowing misclassified example



Leave-one-out cross-validation error (LOOCV)

To assess the robustness of the SVM

$$\text{LOOCV} = \frac{1}{n} \sum_{t=1}^n \text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))),$$

- $(\boldsymbol{\theta}^{-t}, \theta_0^{-t})$ is learnt from the dataset with t^{th} sample left out, i.e., $\mathcal{D}^{-t} = \mathcal{D} \setminus \{\mathbf{x}_t, y_t\}$
- Loss function: $\text{Loss} = \mathbb{1}\{y \neq y'\}$ is zero-one loss

In other words,

$$(\boldsymbol{\theta}^{-t}, \theta_0^{-t}) \in \arg \min_{(\boldsymbol{\theta}, \theta) \in \mathbb{R}^d \times \mathbb{R}} \left\{ \frac{1}{2} \|\boldsymbol{\theta}\|^2 : y_i(\mathbf{x}_i^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall i \in [n] \setminus \{t\} \right\}.$$



Leave-one-out cross-validation (LOOCV) error

If LOOCV is small, the model generalizes well (fits well on new samples)

Proposition 0.5

Let the number of **support vectors** be $N \in \{0, 1, \dots, n\}$. Then

$$\text{LOOCV} \leq \frac{N}{n}.$$

- If the number of support vectors is small, then LOOCV is also small and the SVM works well on new samples



Leave-one-out cross-validation (LOOCV) error

Hint for proof:

- If $\mathbf{x}_t$ is not a support vector, then it cannot influence the solution of the primal SVM.
- Hence,

$$f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, b^{-t})) = y_t, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

and

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) = 0, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

- For any other $\mathbf{x}_t \in \text{SV}$,

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) \leq 1, \quad \forall \mathbf{x}_t \in \text{Support Vectors (SV)}$$



Support Vector Machine (SVM)

- An algorithm that learns the **maximum (geometric) margin linear classifier**
- The solution is **unique**
- Incorporating an **offset** may result in larger geometric margin
- LOOCV to assess the **robustness** of SVM
- For non-linearly separable data: allow misclassified examples by incorporating a **slack**
- Solving SVM: Dual form, sequential minimal optimization (SMO)...
- ◉ Reading: Note of 6-867 (MIT)
Sec 7.2 and Chapter 12 of book “Mathematics for Machine Learning”



Outline

1 Feature engineering

- Polynomial transformation
- Boolean, nominal, ordinal
- Missing values
- Nonlinear transformations

2 Model evaluation and choice

- Generalization of supervised learning algorithms
- Error metric, regularizer, no free lunch!
- Generalization, overfitting
- Data partition: train, test and validation



Q: What is feature engineering?



¹ “Machine Learning and AI via Brain simulation”. Stanford University. Retrieved 2019-08-01

Q: What is feature engineering?

A: Feature engineering is the process of using domain knowledge to

从原始数据中提取对模型有用的特征

extract features (characteristics, properties, attributes) from raw data¹.



¹ “Machine Learning and AI via Brain simulation”. Stanford University. Retrieved 2019-08-01

Q: What is feature engineering?

A: Feature engineering is the process of using domain knowledge to

extract features (characteristics, properties, attributes) from raw data¹.

How?



¹ “Machine Learning and AI via Brain simulation”. Stanford University. Retrieved 2019-08-01

Polynomial regression

- **Learning/Training:**

$$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top)^{-1} \mathbf{y}$$

where

$$\mathbf{P} = \mathbf{P}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m) = \begin{bmatrix} -\mathbf{p}_1^\top & - \\ -\mathbf{p}_2^\top & - \\ \vdots & \\ -\mathbf{p}_m^\top & - \end{bmatrix} \in \mathbb{R}^{m \times \binom{d+p}{p}}$$

- **Prediction/Testing:** Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*.$$



Polynomial transformation

- Generalized Linear Discriminant Function

→ 加高次项

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{1 \leq i < j \leq d} w_{i,j} x_i x_j + \sum_{1 \leq i < j < k \leq d} w_{i,j,k} x_i x_j x_k$$

- Noting that $x_{l,i}$ is the i -th ($1 \leq i \leq d$) component of the l -th ($1 \leq l \leq m$) sample, we can stack this into

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{P}\mathbf{w} = \begin{bmatrix} \mathbf{p}_1^\top \mathbf{w} \\ \vdots \\ \mathbf{p}_m^\top \mathbf{w} \end{bmatrix}$$

and

$$\mathbf{p}_l^\top \mathbf{w} = \begin{bmatrix} 1 & x_{l,1} & \dots & x_{l,d} & \dots & x_{l,i}x_{l,j} & \dots & x_{l,i}x_{l,j}x_{l,k} & \dots \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \\ \vdots \\ w_{i,j} \\ \vdots \\ w_{i,j,k} \\ \vdots \end{bmatrix}$$



Polynomial transformation

- Note that the polynomial matrix

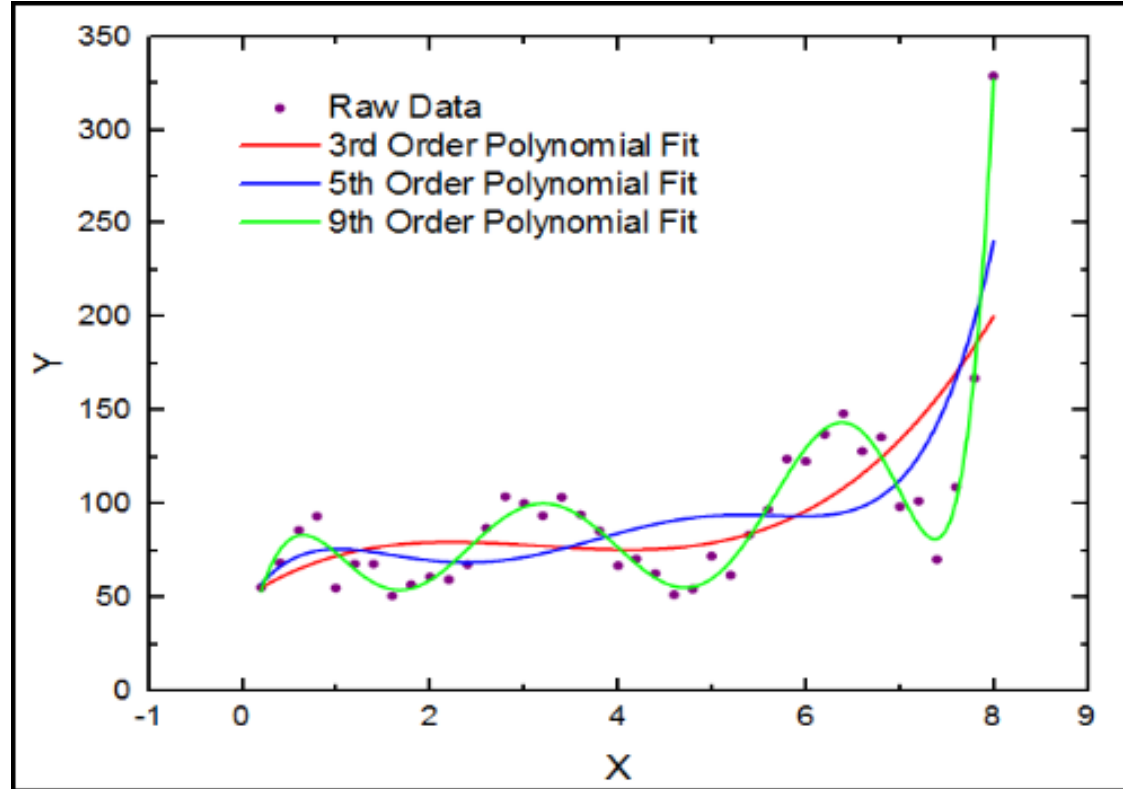
$$\mathbf{P} = \mathbf{P}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m) = \begin{bmatrix} -\mathbf{p}_1^\top & - \\ -\mathbf{p}_2^\top & - \\ \vdots & \\ -\mathbf{p}_m^\top & - \end{bmatrix} \in \mathbb{R}^{m \times \binom{d+p}{p}}$$

is a function of the data samples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$.

- For an d -variable, order- p polynomial, the matrix $\mathbf{P}$ is of size $m \times \binom{d+p}{p}$.
- $d' = \binom{d+p}{p}$ can be far larger than d



Demo 1: polynomial transformation



Source: <https://towardsdatascience.com/polynomial-regression-an-alternative-for-neural-networks-c4bd30fa6cf6/>



Demo 2: polynomial transformation

```
2 import numpy as np
3 from numpy.linalg import inv
4 from numpy.linalg import matrix_rank
5 from sklearn.preprocessing import PolynomialFeatures
6 X = np.array([[1, 1], [-1, 1], [1, -1], [-1, -1]])
7 y = np.array([[1], [-1], [-1], [1]])
8 ## Generate polynomial features
9 order = 2
10 poly = PolynomialFeatures(order)
11 print(poly)
12 P = poly.fit_transform(X)
13 print("matrix P")
14 print(P)
```



Notation: Boolean indicator function

Define

$$\mathbb{1}(\text{statement}) = \begin{cases} 1 & \text{statement is true} \\ 0 & \text{statement is false} \end{cases}$$



Example:

- $\mathbb{1}(1 > 0) = 1$
- $\mathbb{1}(17 < 15) = 0$



Boolean variables

- $\mathcal{X} = \{\text{true}, \text{false}\}$
- Transform to numerical variables:
let $\phi : \mathcal{X} \rightarrow \{0, 1\} : \phi(x) = \mathbb{1}(x)$.



Boolean variables

- $\mathcal{X} = \{ \overset{1}{\text{true}}, \overset{0}{\text{false}} \}$

- Transform to numerical variables:

let $\phi : \mathcal{X} \rightarrow \{0, 1\} : \phi(x) = \mathbb{1}(x)$.

- Example: bank customer dataset

x_1 = with checking account?

x_1 = with saving account?

x_1 = with credit account?



Boolean expressions

- $\mathcal{X} = \{\text{true}, \text{false}\}^2$
 $= \{(\text{true}, \text{true}), (\text{true}, \text{false}), (\text{false}, \text{true}), (\text{false}, \text{false})\}.$
- For $x = (x_1, x_2) \in \mathcal{X}$
- Let $\phi(x) = [\mathbb{1}(x_1), \mathbb{1}(x_2), \mathbb{1}(x_1 \text{ and } x_2), \mathbb{1}(x_1 \text{ or } x_2)]$



Boolean expressions

- $\mathcal{X} = \{\text{true}, \text{false}\}^2$
 $= \{(\text{true}, \text{true}), (\text{true}, \text{false}), (\text{false}, \text{true}), (\text{false}, \text{false})\}.$

- For $x = (x_1, x_2) \in \mathcal{X}$

- Let $\phi(x) = [\mathbb{1}(x_1), \mathbb{1}(x_2), \mathbb{1}(x_1 \text{ and } x_2), \mathbb{1}(x_1 \text{ or } x_2)]$

把逻辑关系当作新特征

- Equivalent: polynomials in $[\mathbb{1}(x_1), \mathbb{1}(x_2)]$ span the same space

- Encodes logical expressions!

$$\begin{aligned} \hookrightarrow \mathbb{1}(x_1 \wedge x_2) &= \mathbb{1}(x_1) \cdot \mathbb{1}(x_2) \\ \mathbb{1}(x_1 \vee x_2) &= \mathbb{1}(x_1) + \mathbb{1}(x_2) - \mathbb{1}(x_1) \cdot \mathbb{1}(x_2) \end{aligned}$$



Nominal values: one-hot encoding

- Nominal data: e.g., $\mathcal{X} = \{\text{apple}, \text{orange}, \text{banana}\}$
- Let

$$\phi(x) = [\mathbb{1}(x = \text{apple}), \mathbb{1}(x = \text{orange}), \mathbb{1}(x = \text{banana})]$$

- One-hot encoding: only one element is non-zero
- Familiar?



Nominal values: one-hot encoding

类别型数据

- Nominal data: e.g., $\mathcal{X} = \{\text{apple}, \text{orange}, \text{banana}\}$
- Let

$$\phi(x) = [\mathbb{1}(x = \text{apple}), \mathbb{1}(x = \text{orange}), \mathbb{1}(x = \text{banana})]$$

- One-hot encoding: only one element is non-zero
- Familiar?

Recall multi-class classification with linear models

- **Extension**: sets,

e.g. you buy an apple and a banana at the same time $\rightarrow$ multi-hot encoding



Nominal values: long tail

- Problem: too many nominal categories
- solution:
 - 1) Cluster the categories by some known ontology
 - (e.g., “apple”, “banana”, “orange” → “fruit”)
 - 2) Lump the least common categories into a single category:
 - “Other” 
- ... Be creative!



Nominal values: look up features!

- ⊙ Why not use other information known about each item?
 - $\mathcal{X} = \{\text{apple, orange, banana}\}$
 - price, calories, weight, ...



Nominal values: look up features!

⊙ Why not use other information known about each item?

- $\mathcal{X} = \{\text{apple, orange, banana}\}$
 - price, calories, weight, ...
- $\mathcal{X} = \{\text{zip code}\}$
 - average income, temperature in July, walk score, %residential, ...
- ...



Ordinal values: real encoding

有序类数据

- Ordinal data: e.g.,

$$\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$$

Let


$$\phi(x) = \begin{cases} 1, & x = \text{Stage I} \\ 2, & x = \text{Stage II} \\ 3, & x = \text{Stage III} \\ 4, & x = \text{Stage IV} \end{cases}.$$

- Default encoding



Ordinal values: real encoding

- $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis
- Use real encoding ϕ to transform ordinal data
- Fit linear model with offset to predict y as $w\phi(x) + b$

共用一个 w

Suppose model predicts a person diagnosed with Stage II cancer will survive 2 more years, and a person diagnosed with Stage I cancer will survive 4 more years.

Q: What is w ? b ?

- A. $b = 6, w = -2$
- B. $b = 2, w = 0$
- C. $b = 6, w = 2$
- D. $b = 0, w = -2$

$$\begin{aligned} 2w + b &= 2 \\ w + b &= 4 \end{aligned} \Rightarrow A$$



Ordinal values: real encoding $\Rightarrow$ 缺点: 会假设类别距离均匀

- $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis
- Use real encoding ϕ to transform ordinal data
- Fit linear model with offset to predict y as $w\phi(x) + b$

Suppose model predicts a person diagnosed with Stage II cancer will survive 2 more years, and a person diagnosed with Stage I cancer will survive 4 more years.

Q: How long does the model predict a person with Stage IV cancer will survive?

- A. 6 years
- B. 2 years
- C. 0 years
- D. -2 years

$$-2 \cdot 4 + 6 \Rightarrow D$$



Ordinal values: Boolean encoding

- Ordinal data: e.g.,

$$\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$$

- Let

$$\phi(x) = [\mathbb{1}(x \geq \text{Stage II}), \mathbb{1}(x \geq \text{Stage III}), \mathbb{1}(x \geq \text{Stage IV})]$$



Ordinal values: Boolean encoding

- $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis
- Define transformation $\phi : \mathcal{X} \rightarrow \mathbb{R}$ as

$$\phi(x) = [\mathbb{1}(x \geq \text{Stage II}), \mathbb{1}(x \geq \text{Stage III}), \mathbb{1}(x \geq \text{Stage IV})]$$

- Fit linear model with offset to predict y as $\mathbf{w}^\top \phi(x) + b$

Suppose model predicts a person diagnosed with Stage II cancer will survive 2 more years, and a person diagnosed with Stage I cancer will survive 4 more years.



Ordinal values: Boolean encoding *比 real-encoding 灵活*

- $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis
- Define transformation $\phi : \mathcal{X} \rightarrow \mathbb{R}$ as

$$y = \begin{cases} b & \text{Stage 1} \\ b + w_1 & \text{Stage 2} \\ b + w_1 + w_2 & \text{Stage 3} \\ \dots & \dots \end{cases}$$

$$\phi(x) = [\mathbb{1}(x \geq \text{Stage II}), \mathbb{1}(x \geq \text{Stage III}), \mathbb{1}(x \geq \text{Stage IV})]$$

- Fit linear model with offset to predict y as $\mathbf{w}^\top \phi(x) + b$

Suppose model predicts a person diagnosed with Stage II cancer will survive 2 more years, and a person diagnosed with Stage I cancer will survive 4 more years.

Q: What is $\mathbf{w}$? b ?

A: $b = 4$, $w_1 = -2$, w_2 and w_3 not determined



Ordinal values: Boolean encoding

- $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis
- Define transformation $\phi : \mathcal{X} \rightarrow \mathbb{R}$ as

e.g. Stage 3: [1 1 0]

$$\phi(x) = [\mathbb{1}(x \geq \text{Stage II}), \mathbb{1}(x \geq \text{Stage III}), \mathbb{1}(x \geq \text{Stage IV})]$$

- Fit linear model with offset to predict y as $\mathbf{w}^\top \phi(x) + b$ 每类都有一个 w_i

Suppose model predicts a person diagnosed with Stage II cancer will survive 2 more years, and a person diagnosed with Stage I cancer will survive 4 more years.

Q: What is $\mathbf{w}$? b ?

A: $b = 4$, $w_1 = -2$, w_2 and w_3 not determined

Q: How long does the model predict a person with Stage IV cancer will survive?

A: Can't say without more information



Missing values

- Remove rows/columns with missing entries
- (For time series) back-fill with most recent observed value



Missing values

- Remove rows/columns with missing entries
- (For time series) back-fill with most recent observed value
- Impute with mean, median, or mode
- Fancier imputation methods (unfortunately NOT covered in this class):
matrix completion, deep learning, ...



Missing values

- Remove rows/columns with missing entries
- (For time series) back-fill with most recent observed value
-  Impute with mean, median, or mode
- Fancier imputation methods (unfortunately NOT covered in this class):
matrix completion, deep learning, ...
- Add new feature: Boolean indicator 1(data is missing)
 - Can detect if missingness is informative
 - Can complement imputation method
 - Can use different indicators for different kinds of missingness
(refused, missing, illegible response, ...)



Poll: missing values

In an ambulance dataset (data taken by instruments on board an ambulance), we want to predict if the patient died. The variable “heart rate” is sometimes missing.

Is missingness

- A. informative?
- B. uninformative?



Poll: missing values

In a weather dataset, the batteries in the instruments occasionally run out before the experimenter can replace them, leaving missing data for eg temperature, humidity, or barometric pressure.

Is missingness

- A. informative?
- B. uninformative?



Talk to your neighbor

Can you think of a dataset in which missing values would be

- A. informative?
- B. uninformative?



Nonlinear transformations

- Sometimes data is easy to predict with a simple but nonlinear relation, e.g.

$$\log(y) = \mathbf{w}^\top \mathbf{x} \text{ equivalent to } y = \exp(\mathbf{w}^\top \mathbf{x})$$

can transform x or (even more important) y

- Familiar?



Nonlinear transformations

- Sometimes data is easy to predict with a simple but **nonlinear** relation, e.g.

$$\log(y) = \mathbf{w}^\top \mathbf{x} \text{ equivalent to } y = \exp(\mathbf{w}^\top \mathbf{x})$$

can transform x or (even more important) y

- Familiar?

log(y)来变线性

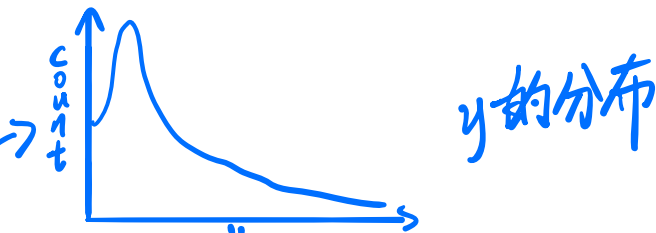
- Nonlinear transforms:

log(y)在右边长尾时用
log, exp, quantile, median, ...

- More systematic ways to handle nonlinearities (**unfortunately NOT covered here**):
 - Deep learning, ...



Nonlinear transformations (optional)



⊙ Hints: your data might benefit from a nonlinear transform

- y is positive and heavy-tailed? try $y \leftarrow \log(y)$
- Residuals $r = y_i - \mathbf{w}^\top \mathbf{x}_i$ are skewed (not normal)
- check with quantile-quantile plot (Q-Q plot).

If skewed, change model.



Location

can be given as

- latitude, longitude
- zip code
- neighborhood, county, state, country

⦿ Can be transformed between these!



Location

can be given as

- latitude, longitude
- zip code
- neighborhood, county, state, country

⊙ Can be transformed between these!

⊙ Which makes sense for your problem?

- Does nearness matter?
- Are there sharp boundaries? e.g., good district for this school but bad for another one
- Are other properties of the location (e.g., mean house price or crime rate) more important?

